LOOKER

&

LOOKER STUDIO

Complete Reference Cheat Sheet

LOOKER / LookML

Data Modelling with LookML — Views, Explores, Dimensions & Measures

Core Concepts

Term	Description
LookML	Looker's modelling language. Sits between your database and users, defining tables, fields and relationships in .lkml files.
Project	A collection of LookML files defining your data model. Maps to one Git repository.
Model	Defines the database connection and available Explores. One model per business domain is common.
Explore	The starting point for querying. Defines which views are joined and how — a pre-built JOIN configuration.
View	Maps to a database table or derived table. Contains dimension and measure definitions.
Dimension	A column-level attribute for filtering, grouping, and display. Never aggregated.
Measure	An aggregation computed at query time. Never stored in the database.
Derived Table	A virtual table defined by SQL or LookML. Can be persisted (PDT) for performance.
PDT	Persistent Derived Table — materialised into the database on a schedule for faster queries.
Datagroup	A cache/refresh policy. Aligns PDT rebuilds and Explore caching with your ETL schedule.

File Structure

```
project/
├── models/
│   └── ecommerce.model.lkml
├── views/
│   ├── orders.view.lkml
│   ├── customers.view.lkml
│   └── products.view.lkml
└── manifest.lkml
```

Model File

```
connection: "my_bigquery_connection"
include: "/views/*.view.lkml"
```

```
explore: orders {
  label: "Orders"
  description: "Order-level data"
```

```
  join: customers {
    type: left_outer
    sql_on: ${orders.customer_id} = ${customers.customer_id} ;;
    relationship: many_to_one
  }
```

```
  join: products {
    type: left_outer
    sql_on: ${orders.product_id} = ${products.product_id} ;;
    relationship: many_to_one
  }
}
```

View File — Structure

```
view: orders {
  sql_table_name: `my_project.my_dataset.orders` ;;

  dimension: order_id {
    type: number
    primary_key: yes
    sql: ${TABLE}.order_id ;;
  }

  dimension_group: order {
    type: time
    timeframes: [date, week, month, quarter, year, day_of_week]
    datatype: date
    sql: ${TABLE}.order_date ;;
  }

  measure: total_revenue {
    type: sum
    sql: ${TABLE}.amount ;;
    value_format_name: gbp
  }

  measure: order_count {
    type: count
    drill_fields: [order_id, customer_id, status]
  }
}
```

Dimensions

Common Types

Type	Description
`string`	Text field
`number`	Numeric field
`yesno`	Boolean — evaluates an expression to YES/NO
`date`	Date only
`time`	Date and time
`tier`	Bins a numeric field into ranges
`duration`	Time between two date dimensions
`zipcode`	US postcode (renders on map)
`location`	Latitude/longitude pair

Examples

```
dimension: is_high_value {
  type: yesno
  sql: ${TABLE}.amount > 1000 ;;
}
```

```
dimension: age_tier {
  type: tier
  tiers: [18, 25, 35, 50, 65]
  style: integer
```

```
  sql: ${age} ;;
}
```

```
dimension: order_band {
  type: string
  sql:
    CASE
      WHEN ${amount} > 1000 THEN 'High'
      WHEN ${amount} > 500  THEN 'Mid'
      ELSE 'Low'
    END ;;
}
```

Dimension Groups (Time)

```
dimension_group: created {
  type: time
  timeframes: [
    raw, time, date, week, month,
    quarter, year, month_name,
    day_of_week, day_of_month, week_of_year
  ]
  datatype: timestamp
  sql: ${TABLE}.created_at ;;
  convert_tz: yes
}
```

Timeframes auto-generate fields like created_date, created_month, created_year etc. Reference them as ${created_date}, ${created_year} in other fields.

Measures

Type	Description
`count`	COUNT(*) — number of rows
`count_distinct`	COUNT(DISTINCT field)
`sum`	SUM(field)
`average`	AVG(field)
`min`	MIN(field)
`max`	MAX(field)
`median`	MEDIAN(field) — warehouse dependent
`number`	Custom calculation using other measures
`percent_of_total`	% of column or row total
`running_total`	Cumulative total down rows
`list`	Comma-separated list of distinct values

Measure Examples

```
measure: avg_order_value {
  type: average
  sql: ${amount} ;;
  value_format_name: decimal_2
}
```

```
measure: uk_revenue {
  type: sum
  sql: ${amount} ;;
```

```
  filters: [country: "UK"]
}
```

```
measure: revenue_per_customer {
  type: number
  sql: ${total_revenue} / NULLIF(${customer_count}, 0) ;;
}
```

Explores — Advanced Options

```
explore: orders {
  group_label: "Sales"
  always_filter: {
    filters: [orders.status: "Complete"]
  }
  conditionally_filter: {
    filters: [orders.order_date: "90 days"]
    unless: [orders.order_id]
  }
  access_filter: {
    field: orders.region
    user_attribute: region
  }
}
```

Relationship	Meaning
`many_to_one`	Many rows in left, one in right — most common (fact to dimension)
`one_to_one`	One row in each table
`one_to_many`	One row in left, many in right — can cause fan-out, use carefully
`many_to_many`	Requires a bridge table to resolve

Derived Tables & PDTs

SQL Derived Table

```
view: customer_ltv {
  derived_table: {
    sql:
      SELECT customer_id,
             SUM(amount)  AS lifetime_value,
             COUNT(*)     AS order_count
      FROM orders
      GROUP BY customer_id
    ;;
  }
}
```

PDT — Persistent Derived Table

```
derived_table: {
  sql: SELECT ... ;;
  datagroup_trigger: daily_refresh   # preferred
  # or: persist_for: "24 hours"
  # or: sql_trigger_value: SELECT MAX(updated_at) FROM orders ;;
}
```

Datagroups

```
datagroup: daily_refresh {
  cron: "0 6 * * *"      # 6am daily
  max_cache_age: "24 hours"
}
```

```
explore: orders {
  persist_with: daily_refresh
}
```

Parameters & Liquid Templating

Parameter

```
parameter: date_granularity {
  type: unquoted
  allowed_value: { label: "Day"   value: "DATE" }
  allowed_value: { label: "Week"  value: "WEEK" }
  allowed_value: { label: "Month" value: "MONTH" }
}
```

```
dimension: dynamic_date {
  sql: DATE_TRUNC(${TABLE}.order_date, {% parameter date_granularity %}) ;;
}
```

Liquid — dynamic SQL

```
dimension: date_trunc {
  sql:
    {% if date_granularity._parameter_value == 'MONTH' %}
      DATE_TRUNC(${TABLE}.order_date, MONTH)
    {% elsif date_granularity._parameter_value == 'WEEK' %}
      DATE_TRUNC(${TABLE}.order_date, WEEK)
    {% else %}
      ${TABLE}.order_date
    {% endif %} ;;
}
```

Access Control

```
access_grant: can_view_salary {
  user_attribute: job_level
  allowed_values: ["manager", "director"]
}
```

```
dimension: salary {
  required_access_grants: [can_view_salary]
  sql: ${TABLE}.salary ;;
}
```

Extends & Refinements

Extend — inherit and override

```
view: orders_extended {
  extends: [orders]
  measure: total_revenue {
    type: sum
```

```
    sql: ${amount} * 1.2 ;;   # override
  }
}
```

Refinement — modify without a new file (prefix +)

```
view: +orders {
  measure: total_revenue_ex_vat {
    type: sum
    sql: ${amount} ;;
  }
}
```

Value Formats

Format Name	Output Example
decimal_0	1,234
decimal_2	1,234.56
usd	$1,234
gbp	£1,234
eur	€1,234
percent_0	12%
percent_2	12.34%

```
value_format: "£#,##0.00"   # custom Excel-style format
```

LookML Best Practices

- One view per table — keep views tightly scoped, avoid cramming unrelated fields
- Always use ${TABLE}.column — never hardcode schema names in SQL blocks
- Mark primary keys — required for symmetric aggregates to work correctly across joins
- Use type: number measures for ratios — never divide inside a sum or average type
- Prefer PDTs with datagroup triggers over persist_for — aligns refresh with your ETL
- Use refinements over extensions for small changes to shared views — keeps the model DRY
- Use always_filter on large table Explores — prevents users querying billions of rows
- Label and describe every field — the field picker is a user's primary interface
- Use required_access_grants for sensitive fields — never rely on hidden: yes for security
- Version control everything — LookML in Git means peer review, rollback and environment promotion

LOOKER STUDIO

Google's free self-service reporting & dashboard tool (formerly Data Studio)

Overview & Key Concepts

Term	Description
Report	The canvas where you build dashboards. Contains pages, charts, and controls.
Data Source	A connection to a database, Google Sheet, BigQuery, etc. Defines available fields.
Dimension	A categorical field used for grouping, labels, and filters (text, date, boolean).
Metric	An aggregated numeric field — the numbers in your charts (sum, count, avg etc.).
Blended Data Source	Combine up to 5 data sources using a JOIN key — like a SQL JOIN in the UI.
Calculated Field	A custom field defined using Looker Studio formula syntax.
Control	Interactive element — date range picker, dropdown filter, slider etc.
Filter	Restricts data shown in a chart or across a page/report.
Segment	A reusable filter condition that can be applied to multiple charts.

Connecting Data Sources

Connector	Notes
Google Sheets	Free, live or extracted. Best for small datasets.
BigQuery	Free connector, billed per query. Best for large datasets.
Google Analytics 4	Native connector. Sampled above 10M sessions.
Google Ads	Native. Pulls campaign, ad group, keyword level data.
Looker (LookML)	Connect directly to a Looker Explore.
PostgreSQL / MySQL	Requires Community Connector or partner connector.
Cloud SQL	GCP managed databases — direct connector available.
CSV Upload	Upload via Google Sheets then connect.

Data sources can be Embedded (report-level, not shareable) or Reusable (shareable across reports). Always use Reusable for team environments.

Chart Types

Chart Type	Best Used For
Scorecard	Single KPI value — headline number
Time Series	Trends over time — line or bar
Bar Chart	Comparing categories
Pie / Donut Chart	Part-to-whole — use sparingly, max 5 slices
Table	Detailed row-level data with sorting
Pivot Table	Cross-tabulation with row and column grouping
Geo Map / Google Maps	Geographic distribution
Scatter Chart	Correlation between two metrics

Bullet Chart	Actual vs target comparison
Treemap	Hierarchical part-to-whole
Gauge	Single metric vs a range
Sankey	Flow between categories

Calculated Fields — Formula Reference

Calculated fields can be created at the Data Source level (available to all charts) or at the Chart level (local only). Formulas use a Google Sheets-like syntax.

Arithmetic

```
Revenue - Cost                          -- subtraction
Revenue / NULLIF(Sessions, 0)           -- safe division
ROUND(Revenue / 1000, 1)                -- round to 1dp
```

Conditional / Logical

```
IF(Revenue > 10000, 'High', 'Low')                          -- if/else
CASE                                                         -- multi-condition
  WHEN Revenue > 10000 THEN 'High'
  WHEN Revenue > 5000  THEN 'Mid'
  ELSE 'Low'
END
```

```
ISNULL(Field)                           -- check for null
IFNULL(Field, 0)                        -- replace null with 0
COALESCE(Field1, Field2, 0)             -- first non-null
```

Text Functions

```
CONCAT(First_Name, ' ', Last_Name)      -- join strings
UPPER(Country)   LOWER(Country)         -- change case
TRIM(Name)                              -- remove whitespace
LEFT(Postcode, 3)                       -- first 3 characters
RIGHT(Code, 2)                          -- last 2 characters
SUBSTR(Text, 2, 5)                      -- substring from pos 2, length 5
LENGTH(Name)                            -- string length
REPLACE(Text, 'old', 'new')             -- replace substring
REGEXP_MATCH(Email, r'.*@gmail\.com')   -- regex match
REGEXP_EXTRACT(URL, r'utm_source=([^&]+)') -- extract via regex
REGEXP_REPLACE(Text, r'[0-9]', '')      -- regex replace
```

Date Functions

```
TODAY()                                 -- current date
NOW()                                   -- current datetime
DATE_DIFF(End_Date, Start_Date, DAY)    -- days between dates
DATE_DIFF(End_Date, Start_Date, WEEK)
DATE_DIFF(End_Date, Start_Date, MONTH)
DATE_ADD(Order_Date, INTERVAL 7 DAY)    -- add 7 days
DATE_TRUNC(Order_Date, MONTH)           -- truncate to month start
DATE_TRUNC(Order_Date, YEAR)
YEAR(Order_Date)                        -- extract year
MONTH(Order_Date)                       -- extract month number
DAY(Order_Date)                         -- extract day
WEEK(Order_Date)                        -- week number
QUARTER(Order_Date)                     -- quarter 1-4
FORMAT_DATETIME('%B %Y', Order_Date)    -- 'January 2024'
PARSE_DATE('%Y%m%d', Date_String)       -- parse text to date
```

Aggregation Functions (in calculated fields)

```
SUM(Revenue)                          -- total
AVG(Revenue)                          -- average
COUNT(Order_ID)                       -- count rows
COUNT_DISTINCT(Customer_ID)           -- count unique
MIN(Order_Date)   MAX(Order_Date)     -- min and max
MEDIAN(Revenue)                       -- median
PERCENTILE(Revenue, 0.75)             -- 75th percentile
STDDEV(Revenue)                       -- standard deviation
```

Type Conversion

```
CAST(Revenue AS TEXT)                 -- number to text
CAST(Date_String AS DATE)             -- text to date
NUMBER(Text_Field)                    -- text to number
TEXT(Number_Field)                    -- number to text
```

Filters & Controls

Control Type	Description
Date Range Control	Lets users pick a date range. Apply to whole page or report.
Drop-down List	Single or multi-select from field values.
Fixed-size List	Visible list of values to select from.
Input Box	Free-text filter — user types a value.
Slider	Numeric range filter.
Checkbox	Boolean toggle filter.
Advanced Filter	Build complex AND/OR filter conditions.

Filter scope: Chart-level (affects only that chart), Page-level (affects all charts on the page), Report-level (affects all pages). Set scope in the filter properties panel.

Filter conditions in calculated fields

```
-- Use CASE for conditional metrics (filtered measures)
SUM(IF(Region = 'North', Revenue, NULL))    -- North revenue only
COUNT(IF(Status = 'Complete', Order_ID, NULL))  -- completed orders
```

Blended Data Sources

Blend up to 5 data sources using one or more join keys. Configured in Resource > Manage blended data. The join type is always LEFT OUTER from the first (primary) source.

Consideration	Detail
Join type	Always LEFT OUTER from primary source — non-matching rows return null
Join keys	Fields with the same name and type in each source
Metrics	Each source contributes its own metrics — aggregated before blending
Performance	Blending is done in Looker Studio, not the database — can be slow on large data
Limit	Maximum 5 data sources per blend

Date Range & Comparison Periods

Enable Comparison Date Range in the Date Range Control or in a chart's date dimension settings to show period-over-period comparisons automatically.

Comparison Option	What It Shows
Previous period	Same length of time immediately before current range
Previous year	Same dates in the prior year
Custom	Define a specific comparison date range

Comparison metric formula

```
-- These are auto-generated when comparison is enabled:
-- metric_name (comparison period)
-- metric_name (absolute change)
-- metric_name (% change)
```

Sharing & Permissions

Permission Level	Can Do
Viewer	View report, interact with controls and filters
Editor	Edit report layout, charts and calculated fields
Owner	Full control — share, delete, change data source

Data credential options: Owner's credentials (viewers see data the owner can see) or Viewer's credentials (viewers only see data they personally have access to — more secure for row-level security).

Embedding reports

```
File > Embed Report > Get embed code
-- Add to website or internal portal via iframe
<iframe src="https://lookerstudio.google.com/embed/reporting/..."
  width="100%" height="600" frameborder="0"></iframe>
```

Useful Tips & Shortcuts

Shortcut / Action	Description
Ctrl + Z / Cmd + Z	Undo
Ctrl + D / Cmd + D	Duplicate selected element
Ctrl + G / Cmd + G	Group selected elements
Hold Shift + click	Multi-select elements
Ctrl + click a chart	Select individual chart while in view mode
Right-click > Make report-level	Promote a filter to affect all pages
View > Theme and Layout	Set report-wide fonts, colours and background
Resource > Manage added data sources	Add, edit or remove data sources
Resource > Manage blended data	Create or edit blended data sources

Looker Studio Best Practices

- Use Reusable data sources for team reports — Embedded sources cannot be shared or reused
- Create calculated fields at the data source level — they become available to all charts and reports
- Use Owner's credentials carefully — all viewers see data the owner can see, including sensitive data

- Limit blended data for large datasets — blending happens in Looker Studio, not the database
- Use BigQuery as your data source for large datasets — it handles scale; Google Sheets does not
- Add a date range control to every dashboard — users expect to filter by time
- Keep reports to one clear purpose per page — avoid cramming too many charts onto one canvas
- Use scorecards with comparison periods for KPI pages — they communicate change at a glance
- Name your data sources and fields clearly — default BigQuery column names are often cryptic
- Use themes for consistent branding — set fonts, colours and logo once in Theme and Layout
- Test with Viewer's credentials before sharing — click View and check what a non-owner sees